

HW-3

Ans. to the ques. no-01

The job of a lexical analyzer is to convert the raw texts into lexemes and tokens. For the designing of a lexical analyzer - there are 2 approaches we can follow -

① High level software automation $\Rightarrow$ Here, we use Lex/Flex to make lexer. It is faster to build, easier to maintain and the only problem it has, is - it depends on the generators runtime.

② Designing a Finite State Machine (FSM) $\Rightarrow$ Based on regular expression's char and symbols, we design a finite state machine, which doesn't necessarily generate an output. It has state transition diagram - which can be written using C/C++, python, assembly or other low level languages.

Ans. to the ques. no-02

Breakdown of what each piece of code does —

① Global declarations & definitions ⇒

Memory storage used across the program. Here, 'in_fp' holds the open file, 'nextChar' holds the single character currently being read, and 'lexeme' is a character array (string) that builds the final word or number.

'#define' → assigns numeric IDs to types of characters (0 for letters, 1 for numbers, 99 for everything else like symbols)

② Main function ⇒

It tries opening the 'input.txt' file and reads the very first character using getChar(), repeatedly calling 'lex()' to grab tokens one by one until it reaches the end of the file (EOF).

Once it's done it closes the .txt file.

III Reading next character (getchar) ⇒

It fetches the next single character from the file using `getc()`.

If it is a letter (a-z, A-Z), it sets 'charclass=LETTER'

If it is a digit (0-9), it sets 'charclass=DIGIT'

If it is a symbol or space, it sets 'charclass=UNKNOWN'

IV Skipping Spaces ⇒

It keeps on calling `getchar()` as long as the current character is a space, tab or a new line.

V Building the current lexeme string ⇒

It appends the characters and attaches it to the end of the 'lexeme' string array - updating its length so we can print the whole word or number together later.

VI Symbol lookup table ⇒

Takes single special symbols like (,), +, -, *, / and assigns their predefined token number.

✓ Core Lexical Analyzer ⇒

It's a token decision engine mainly -
checking what kind of symbol we are
looking at right now and finally
it prints out the result line

*lexer_for_comments.cpp - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main(void) : int

Management

- Projects
- Symbols
- Workspace

```
17 #define DIGIT 1
18 #define UNKNOWN 99
19
20 /* Token codes */
21 #define INT_LIT 10
22 #define IDENT 11
23 #define ASSIGN_OP 20
24 #define ADD_OP 21
25 #define SUB_OP 22
26 #define MULT_OP 23
27 #define DIV_OP 24
28 #define LEFT_PAREN 25
29 #define RIGHT_PAREN 26
30
31 /* Function declarations */
32 void addChar(void);
33 void getChar(void);
34 void getNonBlank(void);
35 int lex(void);
36 int lookup(char ch);
37
38 int main(void) {
39     /* Open input file containing: (sum + 47)/total */
40     if ((in_fp = fopen("D:\\CSE425\\input.txt.txt", "r")) == NULL) {
41         printf("ERROR - cannot open input.txt \n");
42     } else {
43         getChar();
44         do {
45             lex();
46         } while (nextToken != EOF);
47     }
48     return 0;
49 }
50
51 int lookup(char ch) {
52     switch (ch) {
53         case '(':
54             addChar();
55             nextToken = LEFT_PAREN;
56             break;
```

Logs & others

- Code::Blocks
- Search results
- Cccc
- Build log
- Build messages
- CppCheck/Vera++
- CppCheck/Vera++ messages
- Cscope
- Debugger
- DoxyBlocks
- Fortran info
- Close

File	Line	Message
------	------	---------

D:\CSE425\lexer_for_comments.cpp C/C++ Windows (CR+LF) WINDOWS-1252 Line 46, Col 36, Pos 938 Insert Modified Read/Write default

*lexer_for_comments.cpp - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help



Management
Projects Symbols
Workspace

```
*lexer_for_comments.cpp X
50
51 int lookup(char ch) {
52     switch (ch) {
53         case '(':
54             addChar();
55             nextToken = LEFT_PAREN;
56             break;
57         case ')':
58             addChar();
59             nextToken = RIGHT_PAREN;
60             break;
61         case '+':
62             addChar();
63             nextToken = ADD_OP;
64             break;
65         case '-':
66             addChar();
67             nextToken = SUB_OP;
68             break;
69         case '*':
70             addChar();
71             nextToken = MULTI_OP;
72             break;
73         case '/':
74             addChar();
75             nextToken = DIV_OP;
76             break;
77         default:
78             addChar();
79             nextToken = EOF;
80             break;
81     }
82     return nextToken;
83 }
84
85 /* addChar - a function to add nextChar to lexeme */
86 void addChar(void) {
87     if (lexLen <= 95) {
88         lexeme[lexLen++] = nextChar;
89         lexeme[lexLen] = 0;
90     }
91 }
```

Logs & others

* Code::Blocks X Search results X Cccc X Build log X Build messages X CppCheck/Vera++ X CppCheck/Vera++ messages X Cscope X Debugger X DoxyBlocks X Fortran info X Close

File	Line	Message
------	------	---------

D:\CSE425\lexer_for_comments.cpp

C/C++

Windows (CR+LF)

WINDOWS-1252

Line 32, Col 20, Pos 550

Insert

Modified

Read/Write

default

*lexer_for_comments.cpp - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> lookup(char ch): int

Management

- Projects
- Symbols
- Workspace

```
185 /* addChar - a function to add nextChar to lexeme */
186 void addChar(void) {
187     if (lexLen <= 95) {
188         lexeme[lexLen++] = nextChar;
189         lexeme[lexLen] = 0;
190     } else {
191         printf("Error - lexeme is too long \n");
192     }
193 }
194
195 /* getChar - a function to get the next character of input and determine its character class */
196 void getChar(void) {
197     if (nextChar == getc(in_fp) != EOF) {
198         if (isalpha(nextChar))
199             charClass = LETTER;
200         else if (isdigit(nextChar))
201             charClass = DIGIT;
202         else
203             charClass = UNKNOWN;
204     } else {
205         charClass = EOF;
206     }
207 }
208
209 /* getNonBlank - a function to call getChar until it returns a non-whitespace character */
210 void getNonBlank(void) {
211     while (isspace(nextChar))
212         getChar();
213 }
214
215 /* lex - a simple lexical analyzer for arithmetic expressions */
216 int lex(void) {
217     lexLen = 0;
218     getNonBlank();
219
220     switch (charClass) {
221         /* Parse identifiers */
222         case LETTER:
223             addChar();
224             getChar();
```

Logs & others

- Code::Blocks
- Search results
- Cccc
- Build log
- Build messages
- CppCheck/Vera++
- CppCheck/Vera++ messages
- Cscope
- Debugger
- DoxyBlocks
- Fortran info
- Close

File	Line	Message
------	------	---------

D:\CSE425\lexer_for_comments.cpp C/C++ Windows (CR+LF) WINDOWS-1252 Line 61, Col 18, Pos 1227 Insert Modified Read/Write default

*lexer_for_comments.cpp - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> lookup(char ch): int

Management Projects Symbols Workspace

```
*lexer_for_comments.cpp
114
115 /* lex - a simple lexical analyzer for arithmetic expressions */
116 int lex(void) {
117     lexLen = 0;
118     getNonBlank();
119
120     switch (charClass) {
121         /* Parse identifiers */
122         case LETTER:
123             addChar();
124             getChar();
125             while (charClass == LETTER || charClass == DIGIT) {
126                 addChar();
127                 getChar();
128             }
129             nextToken = IDENT;
130             break;
131
132         /* Parse integer literals */
133         case DIGIT:
134             addChar();
135             getChar();
136             while (charClass == DIGIT) {
137                 addChar();
138                 getChar();
139             }
140             nextToken = INT_LIT;
141             break;
142
143         /* Parentheses and operators */
144         case UNKNOWN:
145             lookup(nextChar);
146             getChar();
147             break;
148
149         /* EOF */
150         case EOF:
151             nextToken = EOF;
152             lexeme[0] = '\0';
153             lexeme[1] = '\0';
154             lexeme[2] = '\0';
155             lexeme[3] = '\0';
156             break;
157     }
158
159     printf("Next token is: %d, Next lexeme is %s\n", nextToken, lexeme);
160     return nextToken;
161 }
```

Logs & others

* Code::Blocks x Search results x Cccc x Build log x Build messages x CppCheck/Vera++ x CppCheck/Vera++ messages x Cscope x Debugger x DoxyBlocks x Fortran info x Close

File Line Message

D:\CSE425\lexer_for_comments.cpp

C/C++

Windows (CR+LF)

WINDOWS-1252

Line 61, Col 18, Pos 1227

Insert

Modified

Read/Write

default



D:\CSE425\lexer_for_commen



```
Next token is: 25, Next lexeme is (  
Next token is: 11, Next lexeme is sum  
Next token is: 21, Next lexeme is +  
Next token is: 10, Next lexeme is 47  
Next token is: 26, Next lexeme is )  
Next token is: 24, Next lexeme is /  
Next token is: 11, Next lexeme is total  
Next token is: -1, Next lexeme is EOF
```

```
Process returned 0 (0x0)    execution time : 1.282 s  
Press any key to continue.
```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\HP>

>> cd D:\CSE425

PS D:\CSE425> Measure-Command { .\lexer_for_comments.exe | Out-Null }

Days	: 0
Hours	: 0
Minutes	: 0
Seconds	: 0
Milliseconds	: 450
Ticks	: 4506950
TotalDays	: 5.21637731481481E-06
TotalHours	: 0.000125193055555556
TotalMinutes	: 0.00751158333333333
TotalSeconds	: 0.450695
TotalMilliseconds	: 450.695

PS D:\CSE425> \$process = Start-Process -FilePath ".\lexer_for_comments.exe" -PassThru -NoNewWindow

Next token is: 25, Next lexeme is (

Next

token is: 11, Next lexeme is sum

Next token is: 21, Next lexeme is +

Next token is: 10, Next lexeme is 47

Next token is: 26, Next lexeme is)

Next token is: 24, Next lexeme is /

Next token is: 11, Next lexeme is total

Next token is:PS D:\CSE425> -1, Next lexeme is EOF

\$process.WaitForExit()

PS D:\CSE425> \$memMB = \$process.PeakWorkingSet64 / 1MB

PS D:\CSE425> \$process.PeakWorkingSet64 / 1MB | ForEach-Object { "{0:N2} MB" -f \$_ }

0.00 MB

PS D:\CSE425>

Ans. to the ques. no-03

the same implementation of lexer using Java, is provided below along with the performance matrices —

```
1  import java.io.*;
2
3  public class Lexer {
4      // Character classes
5      private static final int LETTER = 0;
6      private static final int DIGIT = 1;
7      private static final int UNKNOWN = 99;
8
9      // Token codes
10     private static final int INT_LIT = 10;
11     private static final int IDENT = 11;
12     private static final int ASSIGN_OP = 20;
13     private static final int ADD_OP = 21;
14     private static final int SUB_OP = 22;
15     private static final int MULT_OP = 23;
16     private static final int DIV_OP = 24;
17     private static final int LEFT_PAREN = 25;
18     private static final int RIGHT_PAREN = 26;
19     private static final int EOF_TOKEN = -1;
20
21     // Global variables
22     private static int charClass;
23     private static StringBuilder lexeme = new StringBuilder();
24     private static char nextChar;
25     private static int nextToken;
26     private static FileReader fileReader;
27
28     public static void main(String[] args) {
29         try {
30             // Opens input.txt in the current directory
31             fileReader = new FileReader("input.txt");
32             getChar();
33             do {
34                 lex();
35             } while (nextToken != EOF_TOKEN);
```

```
3  public class Lexer {
28      public static void main(String[] args) {
29          try {
35              } while (nextToken != EOF_TOKEN);
36              fileReader.close();
37          } catch (IOException e) {
38              System.err.println("ERROR - cannot open input.txt");
39          }
40      }
41
42      /* lookup - looks up operators and parentheses and returns token code */
43      private static int lookup(char ch) {
44          switch (ch) {
45              case '(':
46                  addChar();
47                  nextToken = LEFT_PAREN;
48                  break;
49              case ')':
50                  addChar();
51                  nextToken = RIGHT_PAREN;
52                  break;
53              case '+':
54                  addChar();
55                  nextToken = ADD_OP;
56                  break;
57              case '-':
58                  addChar();
59                  nextToken = SUB_OP;
60                  break;
61              case '*':
62                  addChar();
63                  nextToken = MULT_OP;
64                  break;
65              case '/':
66                  addChar();
```

```
3 public class Lexer {
102     /* getNonBlank - skips whitespace characters */
103     private static void getNonBlank() {
104         while (Character.isWhitespace(nextChar) && charClass != EOF_TOKEN) {
105             getChar();
106         }
107     }
108
109     /* lex - main lexical analyzer engine */
110     public static int lex() {
111         lexeme.setLength(0); // reset lexeme string
112         getNonBlank();
113
114         switch (charClass) {
115             case LETTER:
116                 addChar();
117                 getChar();
118                 while (charClass == LETTER || charClass == DIGIT) {
119                     addChar();
120                     getChar();
121                 }
122                 nextToken = IDENT;
123                 break;
124
125             case DIGIT:
126                 addChar();
127                 getChar();
128                 while (charClass == DIGIT) {
129                     addChar();
130                     getChar();
131                 }
132                 nextToken = INT_LIT;
133                 break;
134
135             case UNKNOWN:
```

```

1      }
2      nextToken = INT_LIT;
3      break;
4
5  case UNKNOWN:
6      lookup(nextChar);
7      getChar();
8      break;
9  case EOF_TOKEN:
10     nextToken = EOF_TOKEN;
11     lexeme.append("EOF");
12     break;
13 }
14
15 System.out.printf("Next token is: %d, Next lexeme is %s\n", nextToken, lexeme.toString());
16 return nextToken;
17 }
18 }
19

```

```
3 public class Lexer {
43     private static int lookup(char ch) {
44         switch (ch) {
45             case '/':
46                 nextToken = DIV_OP;
47                 break;
48             default:
49                 addChar();
50                 nextToken = EOF_TOKEN;
51                 break;
52         }
53         return nextToken;
54     }
55
56     /* addChar - appends nextChar to the current lexeme */
57     private static void addChar() {
58         lexeme.append(nextChar);
59     }
60
61     /* getChar - gets the next character and sets its character class */
62     private static void getChar() {
63         try {
64             int c = fileReader.read();
65             if (c != -1) {
66                 nextChar = (char) c;
67                 if (Character.isLetter(nextChar))
68                     charClass = LETTER;
69                 else if (Character.isDigit(nextChar))
70                     charClass = DIGIT;
71                 else
72                     charClass = UNKNOWN;
73             } else {
74                 charClass = EOF_TOKEN;
75             }
76         } catch (IOException e) {
```

```
PS E:\Desktop\Java files> $process = Start-Process -FilePath  
"java" -ArgumentList "Lexer" -PassThru -NoNewWindow  
PS E:\Desktop\Java files> $process.WaitForExit()  
Next token is: 25, Next lexeme is (  
Next token is: 11, Next lexeme is sum  
Next token is: 21, Next lexeme is +  
Next token is: 10, Next lexeme is 47  
Next token is: 26, Next lexeme is )  
Next token is: 24, Next lexeme is /  
Next token is: 11, Next lexeme is total  
Next token is: -1, Next lexeme is EOF  
PS E:\Desktop\Java files> $memMB = $process.PeakWorkingSet64  
/ 1MB  
PS E:\Desktop\Java files> Write-Host "Java Memory Usage: $me  
mMB MB"  
Java Memory Usage: 0 MB  
PS E:\Desktop\Java files> |
```

```
Windows PowerShell
PS E:\Desktop\Java files> javac Lexer.java
PS E:\Desktop\Java files> java Lexer
Next token is: 25, Next lexeme is (
Next token is: 11, Next lexeme is sum
Next token is: 21, Next lexeme is +
Next token is: 10, Next lexeme is 47
Next token is: 26, Next lexeme is )
Next token is: 24, Next lexeme is /
Next token is: 11, Next lexeme is total
Next token is: -1, Next lexeme is EOF
PS E:\Desktop\Java files> ^C
PS E:\Desktop\Java files> Measure-Command { java Lexer | Out-Null }

Days           : 0
Hours          : 0
Minutes        : 0
Seconds        : 0
Milliseconds    : 256
Ticks          : 2561075
TotalDays      : 2.96420717592593E-06
TotalHours     : 7.11409722222222E-05
TotalMinutes   : 0.00426845833333333
TotalSeconds   : 0.2561075
TotalMilliseconds : 256.1075
```

Ans. to the ques. no-04

A finite automata is a formal device for recognizing whether a string belongs to a language, - defined by the 5-tuple

$$(Q, \Sigma, \delta, q_0, F)$$

where,

$Q \Rightarrow$ a finite set of states

$\Sigma \Rightarrow$ alphabets allowed as inputs

$\delta \Rightarrow$ transition function.

$q_0 \Rightarrow$ start state

$F \Rightarrow$ set of final states.

For lexical analyzer, each token class (identifier, number, operator, keyword) can be described using regular expression. The regular expression is then converted to NFA.

But NFA is harder to implement using programming languages, so it is later on converted to DFA.

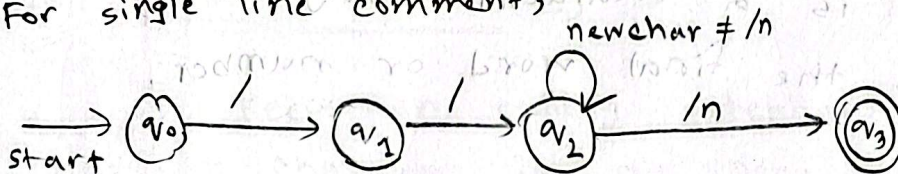
It follows one transition per unit character starting from the start state q_0 . Whenever it reaches an accepting state, records

the longest match found so far as one token, and then it resets to q_0 and continues the cycle.

Ans. to the ques. no-05

For the state diagram to recognize comments in both Java and C programming,

For single line comment,



For block comments,

